

Python E-Course

Module 16 — Capstone Projects

Detailed Notes

Module Overview

Level: Application

Duration: ~2 weeks

Prerequisites: Modules 1-15

What This Module Is

Build, polish, and ship at least one substantial project that exercises everything you've learned. Four short scaffolding lessons cover the how of project building (planning, structure, testing CV pipelines, shipping). The real work lives in the `projects/` folder, which holds 8 self-contained capstone projects you can pick from.

Lessons

#	Lesson	Duration
1	Planning a Computer-Vision Project	25 min
2	CV Pipeline Architecture	25 min
3	Testing Computer-Vision Code	25 min
4	Shipping & Portfolio	25 min

Standalone Projects (in `projects/`)

#	Project	Modules Combined
1	Photo Editor CLI	2-8
2	Document Scanner	5, 9, 11, 12
3	License Plate Detector	9-13
4	Color-Based Object Tracker	4, 10, 12
5	Panorama Stitcher	5, 13
6	Barcode / QR Reader	6, 8, 11
7	Surface Defect Detection	6, 10-12
8	Image Forensics Tool	2, 7, 14

Each project README includes problem statement, ASCII architecture, step-by-step guide, starter code with TODOs, a complete reference solution, test plan, and 3 extension challenges.

Assessment

- Exercises — light warm-ups for the scaffolding lessons.
- Quiz — 10 MCQ on project-building methodology.
- Assignment — the Tier-A capstone specification.
- Project — the Tier-B mini-capstone specification (pick one of the smaller standalone projects).

What's Next

After this module: see [career/career-guide.md](#) and [career/portfolio-ideas.md](#). Run the certification test.

Lesson 1: Planning a Computer-Vision Project

Module: 16 — Capstone

Duration: 25 minutes

Difficulty: Applied

Learning Objectives

- Write a one-page spec for a CV project before any code.
- Decompose a vision pipeline into named stages.
- Pick the smallest v1 that's still meaningful.

Prerequisites

- Modules 1-15.

1. Why This Matters

The biggest failure mode for hobby CV projects is "I'll figure it out as I go." Five minutes of planning beats five hours of refactoring. CV in particular benefits from explicit stage thinking because pipelines are linear and each stage's output can be inspected.

2. Concept

2.1 The spec template

Write before opening an editor:

```
What:      <one sentence>
For:       <who or what use case>
Inputs:    <image source, size, format>
Outputs:   <annotated image / JSON / count / classification>
v1 stages: <bulleted pipeline, 3-7 items>
v1 NOT:    <explicit non-goals>
Success:   <one concrete sentence: "I can detect X with Y precision">
```

Example for a document scanner:

```
What:   Take a phone photo of a tilted document and produce a flat scan.
For:    Personal use — replace dragging out a scanner.
Inputs: Phone photo, 8-12 MP, JPG.
Outputs: Top-down rectangular PNG, b/w-thresholded option.
v1 stages: load → resize → Canny → contours → largest 4-vertex polygon →
perspective warp → adaptive threshold.
v1 NOT: OCR, multi-page, perspective from corners < 50px gap, fancy lighting models.
Success: I can scan 8 of 10 typical phone photos to a readable result, total
processing < 2 seconds.
```

2.2 Decompose into pipeline stages

Every CV project is a sequence of operations. Sketch them as stages with clear inputs/outputs.

Each stage should be:

- Inspectable — you can visualise its input and output.
- Replaceable — swap one implementation for another without touching the rest.
- Testable — assert something about its output given a known input.

2.3 Smallest meaningful v1

Don't try to be robust before you've been complete. Get the walking skeleton:

- Load one specific test image.
- Run all stages.
- Save the result.

Done? Now make it work on 5 more images, then handle edge cases.

2.4 Failure-mode list

Before coding, write down what could go wrong:

- Camera is upside down → handle EXIF rotation.
- Document fills frame → no margin for contours.
- Glare on glossy paper → adaptive threshold fails.
- Background has many edges → wrong contour selected.

Each entry hints at a stage that needs hardening.

2.5 Time-box the v1

Pick a hard limit (e.g. "I'll have v1 working by Friday"). If something's hard to do in v1, defer it explicitly to v2 in the spec.

3. Code Examples

(This lesson is process, not code. Templates instead.)

Template — spec.md

```
# <project name>

## What
<sentence>

## For
<sentence>

## Inputs
- size:
- format:
- source:

## Outputs
- format:
- where saved:
```

```

## v1 stages
1.
2.
3.

## v1 non-goals
-
-

## Success
"I can ..."

```

Template — pipeline sketch (ASCII)

[input] → load → resize → preprocess → detect → annotate → [output]

Template — failure modes

```

| Failure | Stage | Mitigation |
|-----|-----|-----|
| ... | ... | ... |

```

4. Parameter Sensitivity

n/a — process lesson.

5. Common Mistakes

Mistake	What Happens	Fix
Skip the spec	Endless yak-shaving	Write 1 page first
Try to be robust before complete	Stuck on edge cases	Walking skeleton first
No success metric	Never declare done	Write "I can ..." sentence
Ignore failure modes	Brittle code	List them, mitigate explicitly

6. Practice Tasks

- Pick a project from the projects/ folder. Write its spec using the template above.
- Sketch the ASCII pipeline.
- List 5 failure modes and mitigations.

7. Key Takeaways

- Spec first, code second.
- Pipeline = sequence of named stages.
- Walking skeleton before robustness.
- Explicit non-goals are as valuable as goals.

8. What's Next

How to organise the code itself — folder layout, modules, configuration.

Lesson 2: CV Pipeline Architecture

Module: 16 — Capstone

Duration: 25 minutes

Difficulty: Applied

Learning Objectives

- Lay out a small CV project (folders, modules, CLI entry).
- Separate I/O, pipeline stages, and visualisation.
- Add per-stage debug image dumps.

Prerequisites

- Module 16 Lesson 1

1. Why This Matters

A clean structure makes CV pipelines easy to debug. The trick: every stage saves an intermediate image on demand. That makes "why is the output wrong?" a 30-second question instead of a 30-minute one.

2. Concept

2.1 Default layout

```
myproject/
├── README.md
├── pyproject.toml      # or requirements.txt
├── .gitignore
├── myproject/         # the package
│   ├── __init__.py
│   ├── __main__.py   # so `python -m myproject` works
│   ├── cli.py        # argparse + dispatch
│   ├── pipeline.py   # the stages
│   ├── io.py         # load / save / dump
│   └── viz.py        # annotation / overlay helpers
├── samples/          # test images
└── tests/            # smoke tests
```

For very small projects, collapse pipeline.py, viz.py, and io.py into one file. Add structure when it earns itself.

2.2 Pipeline as named functions

Each stage is a function `f(img, **params) -> img`:

```
def preprocess(img):
    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
    return cv2.GaussianBlur(gray, (5, 5), 0)

def detect_edges(gray):
```

```

    return cv2.Canny(gray, 50, 150)

def find_quad(edges):
    contours, _ = cv2.findContours(edges, cv2.RETR_EXTERNAL,
    cv2.CHAIN_APPROX_SIMPLE)
    # ... returns 4 points

```

The pipeline driver chains them and optionally saves intermediates:

```

def run(img, debug_dir=None):
    pre = preprocess(img)
    if debug_dir: cv2.imwrite(f"{debug_dir}/01_pre.png", pre)
    edges = detect_edges(pre)
    if debug_dir: cv2.imwrite(f"{debug_dir}/02_edges.png", edges)
    quad = find_quad(edges)
    ...

```

2.3 Visualisation module

Keep annotation code (rectangles, text overlays, color masks) in viz.py. Don't mix it with the pipeline — your unit tests will thank you.

2.4 Config + defaults

For tunable parameters, use a Config dataclass or simple dict:

```

from dataclasses import dataclass

@dataclass
class Config:
    canny_low: int = 50
    canny_high: int = 150
    min_area: int = 500

```

Pass Config() to run(img, cfg). Easier to test, easier to tune.

2.5 CLI entry

```

# myproject/cli.py
import argparse, cv2
from .pipeline import run, Config

def main():
    p = argparse.ArgumentParser()
    p.add_argument("input")
    p.add_argument("--output", default="result.png")
    p.add_argument("--debug-dir")
    p.add_argument("--canny-low", type=int)
    p.add_argument("--canny-high", type=int)
    args = p.parse_args()

    cfg = Config()
    if args.canny_low is not None: cfg.canny_low = args.canny_low
    if args.canny_high is not None: cfg.canny_high = args.canny_high

```



```

img = cv2.imread(args.input)
out = run(img, cfg, debug_dir=args.debug_dir)
cv2.imwrite(args.output, out)

```

Then `python -m myproject INPUT --debug-dir dump/` writes every stage to `dump/`.

2.6 The "explain this output" workflow

When something looks wrong:

- Re-run with `--debug-dir`.
- Open the numbered intermediate images in order.
- Find the first one that looks wrong.
- Tune that stage's parameters.

This turns vague "doesn't work" into specific "Canny threshold too high."

3. Code Examples

Example 1: Minimal pipeline structure

```

# project/pipeline.py
from dataclasses import dataclass
import cv2

@dataclass
class Config:
    canny_low: int = 50
    canny_high: int = 150

def preprocess(bgr):
    gray = cv2.cvtColor(bgr, cv2.COLOR_BGR2GRAY)
    return cv2.GaussianBlur(gray, (5, 5), 0)

def detect_edges(gray, cfg):
    return cv2.Canny(gray, cfg.canny_low, cfg.canny_high)

def run(bgr, cfg=Config(), debug_dir=None):
    pre = preprocess(bgr)
    edges = detect_edges(pre, cfg)
    if debug_dir:
        from pathlib import Path
        Path(debug_dir).mkdir(parents=True, exist_ok=True)
        cv2.imwrite(f"{debug_dir}/01_pre.png", pre)
        cv2.imwrite(f"{debug_dir}/02_edges.png", edges)
    return edges

```

Example 2: Tiny CLI wrapper

```

# project/cli.py
import argparse, cv2
from .pipeline import run, Config

def main():

```

```

p = argparse.ArgumentParser()
p.add_argument("input")
p.add_argument("--output", default="out.png")
p.add_argument("--debug-dir")
args = p.parse_args()
img = cv2.imread(args.input)
out = run(img, Config(), debug_dir=args.debug_dir)
cv2.imwrite(args.output, out)

if __name__ == "__main__":
    main()

```

Example 3: Test a single stage in isolation

```

# tests/test_preprocess.py
import numpy as np
from project.pipeline import preprocess

def test_preprocess_output_shape():
    img = np.zeros((100, 200, 3), dtype=np.uint8)
    out = preprocess(img)
    assert out.shape == (100, 200)
    assert out.dtype == np.uint8

```

4. Parameter Sensitivity

n/a — architecture lesson.

5. Common Mistakes

Mistake	What Happens	Fix
One giant function	Impossible to debug	Split into stages
Mix viz with logic	Untestable	Separate viz.py
Hardcoded params	Re-edit code to tune	Config dataclass
No debug-dir	Have to add cv2.imwrite ad-hoc	Build it in from v1

6. Practice Tasks

- Lay out the recommended package structure for one project from projects/.
- Refactor the project's starter code into 3 named stages.
- Add a --debug-dir flag that dumps each stage.

7. Key Takeaways

- Pipeline = sequence of small named stages.
- Config dataclass over loose globals.
- Add --debug-dir from the start.
- Separate I/O, logic, and visualisation.

8. What's Next

Testing — how to write meaningful tests for code where the "correct" output is fuzzy.

Lesson 3: Testing Computer-Vision Code

Module: 16 — Capstone

Duration: 25 minutes

Difficulty: Applied

Learning Objectives

- Write smoke tests for CV pipelines.
- Use synthesised fixtures (small drawn images) for stage-level tests.
- Use PSNR/SSIM thresholds for regression checks.
- Set up a sample-image evaluation harness.

Prerequisites

- Module 16 Lesson 2

1. Why This Matters

Testing CV code is awkward — there's rarely a single "correct" output. But you can still pin down properties: shape, dtype, score thresholds, regression vs a known-good output. Five tests cover the cases that actually break.

2. Concept

2.1 Three test layers

- Smoke tests — "does it run without crashing on a typical input?"
- Stage tests — "given a known input, does this stage produce the expected output?" (assert shape, type, count of detections).
- End-to-end regression — "is the output close enough (PSNR/SSIM) to a saved reference?"

2.2 Synthetic fixtures

Build tiny images programmatically — much faster than loading samples:

```
import numpy as np, cv2
def make_square_doc():
    img = np.full((400, 400), 250, dtype=np.uint8)
    cv2.rectangle(img, (50, 50), (350, 350), 100, -1)
    return img
```

Use these for stage-level tests where you control everything.

2.3 Pytest pattern

```
import numpy as np
from myproject.pipeline import detect_quad

def test_detect_quad_finds_4_corners():
    img = make_square_doc()
    quad = detect_quad(img)
```

```
assert quad is not None
assert quad.shape == (4, 2)
```

2.4 PSNR-based regression

For pipelines where the result is an image (deblur, restore, recolor):

```
from skimage.metrics import peak_signal_noise_ratio
def test_regression():
    img = cv2.imread("samples/in.png")
    expected = cv2.imread("samples/expected.png")
    actual = run(img)
    assert peak_signal_noise_ratio(expected, actual, data_range=255) > 30
```

Set the threshold based on what you observed during development.

2.5 Property-based tests

Useful properties to check on outputs:

- Mask is uint8 in {0, 255}.
- Detected count > 0.
- Bounding box inside image bounds.
- Centroid coordinates positive.
- Output image same size as input.

2.6 Visual review folder

Even with tests, keep a small folder of samples/before/ + samples/after/ that you spot-check after each commit. CV bugs that pass numerical tests often catch your eye instantly.

2.7 CI considerations

If putting this in CI:

- Pin OpenCV version (small algorithm changes between versions can flip outputs).
- Set a random seed in tests that involve noise.
- Use small synthetic images, not 4K samples.

3. Code Examples

Example 1: Smoke test

```
# tests/test_smoke.py
import numpy as np
from myproject.pipeline import run

def test_smoke_runs():
    img = np.full((300, 400, 3), 128, dtype=np.uint8)
    out = run(img)
    assert out is not None
```

Example 2: Stage test with synthetic fixture

```
import numpy as np, cv2
from myproject.pipeline import find_blobs

def make_two_blobs():
    img = np.zeros((200, 400), dtype=np.uint8)
    cv2.circle(img, (100, 100), 30, 255, -1)
    cv2.circle(img, (300, 100), 40, 255, -1)
    return img

def test_finds_two_blobs():
    blobs = find_blobs(make_two_blobs())
    assert len(blobs) == 2
```

Example 3: PSNR-based regression

```
import cv2
from skimage.metrics import peak_signal_noise_ratio
from myproject.pipeline import run

def test_regression_quality():
    inp = cv2.imread("samples/01_in.jpg")
    expected = cv2.imread("samples/01_expected.png")
    actual = run(inp)
    assert peak_signal_noise_ratio(expected, actual, data_range=255) > 28
```

Example 4: Eval harness on a folder of samples

```
import cv2
from pathlib import Path
from myproject.pipeline import run

def evaluate(folder):
    folder = Path(folder)
    results = []
    for path in sorted(folder.glob("*.jpg")):
        img = cv2.imread(str(path))
        out = run(img)
        cv2.imwrite(str(folder / "out" / path.name), out)
        results.append(path.name)
    print(f"processed {len(results)} images")

if __name__ == "__main__":
    evaluate("samples")
```

4. Parameter Sensitivity

n/a — process lesson.

5. Common Mistakes

Mistake	What Happens	Fix
No tests at all	Refactor with fear	Add at least smoke test
Test with random images	Flaky on order changes	Use synthetic + seed
Test against pixel-exact	Tiny OpenCV changes break	Use PSNR/SSIM thresholds

output	things	
Test only happy path	Edge cases ship broken	Add "empty input" / "no detection" tests

6. Practice Tasks

- Write a smoke test for one project from projects/.
- Write a stage test using a synthetic fixture.
- Add a PSNR-based regression test for a denoising pipeline.

7. Key Takeaways

- Smoke + stage + regression = 3 layers.
- Synthetic fixtures > samples for stage tests.
- PSNR/SSIM thresholds for fuzzy outputs.
- Keep a visual review folder.

8. What's Next

The final lesson: shipping the project (README, demo gif, releases).

Lesson 4: Shipping & Portfolio

Module: 16 — Capstone

Duration: 25 minutes

Difficulty: Applied

Learning Objectives

- Write a README that demos a CV project clearly.
- Capture before/after images for the README.
- Push to GitHub and tag a release.
- Talk about a CV project in an interview.

Prerequisites

- Module 16 Lesson 3

1. Why This Matters

A great CV project no one understands is invisible. CV projects in particular live or die on visuals — a before/after image in the README does more than any paragraph. Spend the last 10% on packaging; it pays back hugely.

2. Concept

2.1 README structure for a CV project

```
# Project Name

One-line tagline.

## Demo

[BEFORE → AFTER image]
[GIF if the project handles video]

## Features
- bullet 1
- bullet 2

## Install
$ python -m venv .venv
$ source .venv/bin/activate
$ pip install -r requirements.txt

## Use
$ python -m project INPUT --output OUT.png
[more examples]

## How it works
- stage 1: ...
```

```
- stage 2: ...
```

```
## Limitations
```

```
- ...  
- ...
```

```
## License
```

```
MIT
```

The "How it works" section is what differentiates a CV project on a portfolio — it shows you understand your pipeline, not just stitched library calls.

2.2 The before/after image

Compose a single PNG with a small dividing line:

```
import cv2, numpy as np  
before = cv2.imread("samples/in.jpg")  
after = cv2.imread("samples/out.png")  
divider = np.full((before.shape[0], 4, 3), 255, dtype=np.uint8)  
combo = np.hstack([before, divider, after])  
cv2.imwrite("demo.png", combo)
```

Show that on top of the README. Reviewers will see the result before reading a word.

2.3 Animated GIFs

For video / live projects: record a screen capture (15-30 s), reduce to GIF with ffmpeg:

```
ffmpeg -i demo.mp4 -vf "fps=10,scale=640:-1" demo.gif
```

GitHub renders inline.

2.4 Limitations section

A short, honest list of what your project doesn't handle ("works only on landscape orientation", "fails under glare", "max 1 document per image") is more impressive than over-claiming. Senior engineers read these to gauge maturity.

2.5 Releases and tags

When v1 works:

```
git tag -a v0.1.0 -m "First release"  
git push origin v0.1.0
```

On GitHub: Releases → Draft new release → pick the tag → list features. Even small projects benefit from a tagged release — it's a milestone you can reference later.

2.6 Talking about a CV project in an interview

Pattern:

- Problem in one sentence: "Phone photos of documents look tilted; I built a tool that flattens them."

- Stack and stages: "OpenCV, scikit-image. Canny → contour → largest 4-vertex polygon → perspective warp → adaptive threshold."
- One thing you learned: "Adaptive threshold beats Otsu when lighting is uneven — I learned that the hard way after Otsu kept failing on documents shot near windows."
- Limitations + next steps: "Doesn't handle multi-page; corners < 50 px from frame edges fail. Next: train a small UNet for corner detection to handle those."

This 60-second answer tells a reviewer you can plan, build, evaluate, and improve.

2.7 Ship checklist

- [] README with demo image at the top.
- [] Install + Use sections work as-is on a clean machine.
- [] Tests pass (even just smoke).
- [] .gitignore excludes samples that aren't yours, .venv, __pycache__.
- [] LICENSE (MIT is fine for personal projects).
- [] One tagged release.

3. Code Examples

Example 1: Build a side-by-side demo image

```
import cv2, numpy as np

before = cv2.imread("samples/01_in.jpg")
after = cv2.imread("samples/01_out.png")
# Match heights (if needed) before stacking
h = min(before.shape[0], after.shape[0])
before = cv2.resize(before, (int(before.shape[1] * h / before.shape[0]), h))
after = cv2.resize(after, (int(after.shape[1] * h / after.shape[0]), h))
divider = np.full((h, 4, 3), 255, dtype=np.uint8)
combo = np.hstack([before, divider, after])
cv2.imwrite("docs/demo.png", combo)
```

Example 2: Generate a 4-up grid for the README

```
import cv2, numpy as np

imgs = [cv2.imread(p) for p in
        ["samples/in1.jpg", "samples/in2.jpg",
         "samples/out1.png", "samples/out2.png"]]
top = np.hstack(imgs[:2])
bottom = np.hstack(imgs[2:])
grid = np.vstack([top, bottom])
cv2.imwrite("docs/grid.png", grid)
```

Example 3: requirements.txt example

```
opencv-python>=4.5
scikit-image>=0.20
numpy>=1.23
Pillow>=9.0
```

Example 4: Tag and release (shell)

```
git add .  
git commit -m "v0.1.0: working document scanner"  
git tag -a v0.1.0 -m "First release"  
git push origin main --tags
```

4. Parameter Sensitivity

n/a — process lesson.

5. Common Mistakes

Mistake	What Happens	Fix
No demo image	Reviewer scrolls past	Demo at the very top
README walls of text	Skipped	Use sections; lead with visuals
Hide limitations	Looks oblivious	Honest "Limitations" section
No tagged release	Project feels in-progress	Tag v0.1.0 the moment v1 works

6. Practice Tasks

- Write a README for one of the projects/ projects using the template.
- Build a side-by-side demo PNG.
- Tag v0.1.0 once your v1 works.

7. Key Takeaways

- README leads with a before/after image.
- "How it works" beats library lists for CV.
- Honest "Limitations" wins respect.
- Tag releases; record GIFs for video projects.

8. What's Next

End of Module 16. Pick a project from projects/ and ship it.

Module 16 — Exercises

These are light scaffolding tasks. The real work is in the assignment and the projects/ directory.

16.1 Planning

- (E) Write a one-page spec for any project from projects/ using the template.
- (M) Sketch its pipeline as an ASCII diagram with named stages.
- (H) List 5 failure modes for that project with mitigations.

16.2 Architecture

- (E) Lay out the recommended package directory for that project.
- (M) Refactor the starter code into 3 named stages.
- (H) Add a --debug-dir flag that dumps each stage as a PNG.

16.3 Testing

- (E) Write a smoke test that runs the pipeline on a synthetic image.
- (M) Write a stage-level test using a synthetic fixture (e.g. a drawn square).
- (H) Add a PSNR-based regression test against a saved expected output.

16.4 Shipping

- (E) Write a README for your chosen project using the lesson-4 template.
- (M) Build a side-by-side demo PNG and embed it at the top of the README.
- (H) Tag v0.1.0 on the repo and write release notes.

Module 16 — Quiz

10 MCQs on capstone methodology.

Q1

The first artefact in a CV project should be: A. the README B. one-page spec C. starter code D. unit tests

Answer: B (L1)

Q2

"Walking skeleton" means: A. fastest implementation B. smallest version that runs end-to-end C. final UI shell D. README

Answer: B (L1)

Q3

Each pipeline stage should be: A. inline in one function B. inspectable, replaceable, testable C. written in C D. parallelised

Answer: B (L2)

Q4

A --debug-dir flag should: A. enable verbose logging B. dump each stage's image to disk C. open a GUI D. profile timings

Answer: B (L2)

Q5

For tunable parameters, prefer: A. globals B. Config dataclass / dict C. environment variables D. hardcoded constants

Answer: B (L2)

Q6

For testing image outputs whose pixel-exact value is unstable: A. assertEquals B. PSNR/SSIM thresholds C. md5 hash D. visual review only

Answer: B (L3)

Q7

A synthetic fixture for tests should: A. be a 10 MP photo B. be drawn programmatically; small and reproducible C. live on disk D. be random

Answer: B (L3)

Q8

What goes at the very top of a CV project README? A. install instructions B. a before/after demo image C. licence D. authors

Answer: B (L4)

Q9

A "Limitations" section is: A. discouraged — looks bad B. helpful — shows engineering maturity C. only for academic projects D. optional

Answer: B (L4)

Q10

When v1 of a project works, you should: A. start v2 immediately B. tag a release (e.g., v0.1.0) and stop to polish docs C. discard and rewrite D. wait for feedback

Answer: B (L4)

Module 16 — Tier-A Capstone Assignment

Estimated time: 12-20 hours

Combines: All of Modules 1-15

Brief

Pick one of the 8 standalone projects in projects/, build it end-to-end, and ship it. The goal isn't "use every module" — it's "build something good enough to put on a CV."

Required deliverables

For the project you choose:

- Package layout per Module 16 Lesson 2 (myproject/__main__.py, pipeline.py, cli.py, viz.py, tests/).
- pyproject.toml or requirements.txt with pinned versions.
- README following the lesson-4 template:
- Title + one-line tagline.
- Before/after demo image at the very top.
- Features bullet list.
- Install + Use sections.
- "How it works" (pipeline stages explained).
- Limitations section.
- LICENSE.
- CLI: `python -m myproject INPUT [--output] [--debug-dir] [project-specific flags]`.
- Tests: at minimum 1 smoke + 2 stage tests; ideally a regression test.
- Sample images in samples/ (use the starter images or your own).
- Tagged release (v0.1.0 once it works).

Recommended workflow

- Read the project's projects/XX-name/README.md to understand the requirements.
- Write a one-page spec.md in your repo (Lesson 1 template).
- Build the walking skeleton (load → core stage → save) on one test image.
- Add the remaining stages one at a time.
- Run on 5 more sample images; fix what breaks.
- Add tests, README, demo image.
- Tag.

Constraints

- Use OpenCV, NumPy, scikit-image, matplotlib, Pillow as needed.
- A third-party CV library beyond these (e.g. pyzbar for QR) is fine; cite it in README.
- No deep learning — this is a classical CV course. (You can add a DL stage in v2.)

Grading Rubric

Criterion	Weight
Walking skeleton works	20%
All required features	30%
Code organisation	15%
Tests pass	10%
README + demo image	15%
Tagged release	5%
Style / clean code	5%

Submission

A GitHub repo URL (or zip), with:

- The package
- Tests
- README + sample images
- Tagged release
- A short (3-5 min) screencast or written walkthrough

Module 16 — Tier-B Mini-Capstone Project

Estimated time: 4-8 hours

Combines: Selected modules per the chosen project.

Brief

Build a shorter version of any project from projects/ — implement the core pipeline only, skip extension challenges. Use this for a quick portfolio piece if you don't have time for the full Tier-A capstone.

Required Deliverables

- Single-file Python script in your repo (no package structure required at this level).
- One README.md with:
 - 2-paragraph description.
 - Install one-liner.
 - Use one-liner.
 - Before/after demo image.
- Sample images demonstrating it working on at least 3 inputs.

Suggested Mini-Capstones (lighter scope of each project)

Pick	Scope reduction
Photo Editor CLI	Just 3 ops (brightness, contrast, blur) + save
Document Scanner	Manual 4-corner click (not auto-detect) + warp + save
License Plate Detector	Locate plate region only; don't read characters
Color Object Tracker	Single image (not video); just bounding box around colored object
Panorama Stitcher	Two pre-aligned images (skip Lowe + RANSAC reliability)
Barcode/QR Reader	Direct pyzbar.decode on input; no preprocessing
Defect Detection	One known good vs one inspect image; threshold absdiff
Image Forensics	Just ELA (Error Level Analysis), nothing else

Constraints

- Use OpenCV / NumPy / scikit-image / matplotlib / Pillow.
- A single .py file is fine; no need for a package.
- README must lead with a demo image.

Grading Rubric

Criterion	Weight
-----------	--------

Script runs as-described	40%
Demo image in README	20%
Quality of result	20%
Code clarity	10%
Sample inputs included	10%

Submission

A GitHub repo or zip with the script, README, and samples/ folder.